```
    let ns = ''
  else
    let ns = M.name
  end
end

loop do
  # Look for a regular file.
  for dir in autoload_paths
    if the file "#{dir}/#{ns.underscore}/c.rb" exists
      load/require "#{dir}/#{ns.underscore}/c.rb"

      if C is now defined
        return
      else
        raise LoadError
      end
    end
  end

  # Look for an automatic module.
  for dir in autoload_paths
    if the directory "#{dir}/#{ns.underscore}/c" exists
      if ns is an empty string
        let C = Module.new in Object and return
      else
        let C = Module.new in ns.constantize and return
      end
    end
  end

  if ns is empty
    # We reached the top-level without finding the constant.
    raise NameError
  else
    if C exists in any of the parent namespaces
      # Qualified constants heuristic.
      raise NameError
    else
      # Try again in the parent namespace.
      let ns = the parent namespace of ns and retry
    end
  end
end
```

7 require_dependency

Constant autoloading is triggered on demand and therefore code that uses a certain constant may have it already defined or may trigger an autoload. That depends on the execution path and it may vary between runs.

There are times, however, in which you want to make sure a certain constant is known when the execution reaches some code. `require_dependency` provides a way to load a file using the currentloading mechanism, and keeping track of constants defined in that file as if they were autoloaded to have them reloaded as needed.
`require_dependency` is rarely needed, but see a couple of use-cases in Autoloading and STI andWhen Constants aren't Triggered.
Unlike autoloading, `require_dependency` does not expect the file to define any particular constant. Exploiting this behavior would be a bad practice though, file and constant paths should match.